

Git Challenges

1. **Resolve Merge Conflicts**

- Create a merge conflict intentionally (two users editing the same line).
- Resolve the conflict and push the changes.

Objective

To understand how Git handles merge conflicts by intentionally creating a conflict when two branches modify the same line of a file differently, and then resolving the conflict and completing the merge.

Steps to Implement

1. Initialize a Git repository and create the initial commit.

2. Create a file named app.txt with initial content:

```
echo "Application Version: 1.0" > app.txt
```

3. Add and commit the file:

```
git add app.txt
```

```
git commit -m "Add application version file"
```

4. Create and switch to a new branch named developer1:

```
git switch -c developer1
```

5. Modify the same line in app.txt:

```
echo "Application Version: 2.0 - Developer1" > app.txt
```

6. Add and commit the changes:

```
git add app.txt
```

```
git commit -m "Developer1 updates application version"
```

7. Switch back to the main branch:

```
git switch main
```

8. Modify the same line differently:

```
echo "Application Version: 3.0 - Main" > app.txt
```

9. Add and commit the changes:

```
git add app.txt
```

```
git commit -m "Main updates application version"
```

10. Merge the developer1 branch into main:

```
git merge developer1
```

11. Git detects a merge conflict because both branches modified the same line.

12. Check the conflicted file:

```
cat app.txt
```

13. Resolve the conflict by editing the file and keeping the required final content:

```
echo "Application Version: 3.0 - Main and Developer1 changes merged" >
```

```
app.txt
```

14. Stage the resolved file:

```
git add app.txt
```

15. Complete the merge by creating a commit:

```
git commit -m "Resolve merge conflict"
```

16. Verify the commit history:

```
git log --oneline --graph --all
```

Result

The merge conflict was successfully created by modifying the same line differently in two branches. The conflict was manually resolved, staged, and committed successfully.

```

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git remote -v
origin https://github.com/chandrashekar0532/git-practice.git (fetch)
origin https://github.com/chandrashekar0532/git-practice.git (push)

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git remote add origin https://github.com/chandrashekar0532/git-challenges.git
error: remote origin already exists.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git remote remove origin

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git remote add origin https://github.com/chandrashekar0532/git-challenges.git

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push -u origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (18/18), 1.36 KiB | 348.00 KiB/s, done.
Total 18 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/chandrashekar0532/git-challenges.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push origin developer1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'developer1' on GitHub by visiting:
remote:   https://github.com/chandrashekar0532/git-challenges/pull/new/developer1
remote:
To https://github.com/chandrashekar0532/git-challenges.git
 * [new branch]      developer1 -> developer1

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$

```

2. Recover Deleted Branch

- Delete a local branch and then recover it using the reflog.

Recover Deleted Branch

Create branch

git switch -c recovery-branch

echo "Important recovery data" > recovery.txt

git add recovery.txt

git commit -m "Add recovery data"

Go back to master:

git switch master

git branch -D recovery-branch

git branch

Find deleted branch commit

git reflog

HEAD@{...}: commit: Add recovery data

26fd4a8

Recover branch

git switch -c recovery-branch 26fd4a8

git branch

ls

recovery.txt should be back

MINGW64:/f/git-practice

```
HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git switch -c recovery-branch
Switched to a new branch 'recovery-branch'

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ echo "Important recovery data" > recovery.tx

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git add recovery.tx
warning: in the working copy of 'recovery.tx', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git commit -m "Add recovery data"
[recovery-branch c5d9d25] Add recovery data
1 file changed, 1 insertion(+)
create mode 100644 recovery.tx

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git switch master
fatal: invalid reference: master

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git branch -D recovery-branch
error: Cannot delete branch 'recovery-branch' checked out at 'F:/Git-practice'

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git branch
  developer1
  main
* recovery-branch

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git switch master
fatal: invalid reference: master

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git branch -D recovery-branch
Deleted branch recovery-branch (was c5d9d25).

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git reflog
224202a (HEAD -> main, origin/main) HEAD@{0}: checkout: moving from recovery-branch to main
c5d9d25 HEAD@{1}: commit: Add recovery data
224202a (HEAD -> main, origin/main) HEAD@{2}: checkout: moving from main to recovery-branch
224202a (HEAD -> main, origin/main) HEAD@{3}: Branch: renamed refs/heads/master to refs/heads/main
224202a (HEAD -> main, origin/main) HEAD@{5}: commit: first commit
122f0bb HEAD@{6}: commit (merge): Resolve merge conflict
0b9104e HEAD@{7}: commit: Update readme in master
ad9b7d5 HEAD@{8}: checkout: moving from developer1 to master
8668219 (origin/developer1, developer1) HEAD@{9}: commit: Update readme in developer1
ad9b7d5 HEAD@{10}: checkout: moving from master to developer1
ad9b7d5 HEAD@{11}: commit: Initial commit
```

```

developer1
main
* recovery-branch

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git switch master
fatal: invalid reference: master

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git branch -D recovery-branch
Deleted branch recovery-branch (was c5d9d25).

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git reflog
224202a (HEAD -> main, origin/main) HEAD@{0}: checkout: moving from recovery-branch to main
c5d9d25 HEAD@{1}: commit: Add recovery data
224202a (HEAD -> main, origin/main) HEAD@{2}: checkout: moving from main to recovery-branch
224202a (HEAD -> main, origin/main) HEAD@{3}: Branch: renamed refs/heads/master to refs/heads/main
224202a (HEAD -> main, origin/main) HEAD@{5}: commit: first commit
122f0bb HEAD@{6}: commit (merge): Resolve merge conflict
0b9104e HEAD@{7}: commit: Update readme in master
ad9b7d5 HEAD@{8}: checkout: moving from developer1 to master
8668219 (origin/developer1, developer1) HEAD@{9}: commit: Update readme in developer1
ad9b7d5 HEAD@{10}: checkout: moving from master to developer1
ad9b7d5 HEAD@{11}: commit: Initial commit
f62523c HEAD@{12}: commit (initial): these are added in my local machine

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git checkout -b recovery-branch <commit_hash>
bash: syntax error near unexpected token 'newline'

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git checkout -b recovery-branch 224202a
Switched to a new branch 'recovery-branch'

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git push origin recovery-branch
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (18/18), 1.36 KiB | 232.00 KiB/s, done.
Total 18 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), done.
remote:
remote: Create a pull request for 'recovery-branch' on GitHub by visiting:
remote:   https://github.com/chandrashekar0532/git-challenges/pull/new/recovery-branch
remote:
To https://github.com/chandrashekar0532/git-challenges.git
 * [new branch]   recovery-branch -> recovery-branch

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ |

```

3. Undo Wrong Push

- Push a wrong commit to GitHub, then undo it without losing history.

echo "BAD CHANGE" > wrong.txt

git add wrong.txt

git commit -m "Add wrong change"

git push origin main

git log --oneline

Revert the wrong commit

git log --oneline

git revert abc1234

Git may open an editor.

Save the default revert message.

git push origin main

Production recommendation

git revert

git reset --hard

```
HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ echo "BAD CHANGE" > wrong.txt

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ ls
README.md a b c d f g git git-challenges/ github github-task/ h i readme.txt wrong.txt

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git add wrong.txt
warning: in the working copy of 'wrong.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git commit -m "Add wrong change"
[recovery-branch 32beb31] Add wrong change
1 file changed, 1 insertion(+)
create mode 100644 wrong.txt

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git remote -v
origin https://github.com/chandrashekar0532/git-challenges.git (fetch)
origin https://github.com/chandrashekar0532/git-challenges.git (push)

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git push origin main
To https://github.com/chandrashekar0532/git-challenges.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/chandrashekar0532/git-challenges.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git push origin developer1
To https://github.com/chandrashekar0532/git-challenges.git
 ! [rejected]        developer1 -> developer1 (fetch first)
error: failed to push some refs to 'https://github.com/chandrashekar0532/git-challenges.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (recovery-branch)
$ git remote add origin https://github.com/chandrashekar0532/git-challenges.git
error: remote origin already exists.
```

```

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git remote add origin https://github.com/chandrashekar0532/git-challenges.git
error: remote origin already exists.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git push origin master
error: src refspec master does not match any
error: failed to push some refs to 'https://github.com/chandrashekar0532/git-challenges.git'

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git branch
  developer1
* main
* recovery-branch

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 3.51 KiB | 156.00 KiB/s, done.
From https://github.com/chandrashekar0532/git-challenges
* branch      main      -> FETCH_HEAD
  224202a..adbe9f1  main    -> origin/main
Updating 224202a..adbe9f1
Fast-forward

```

```

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (recovery-branch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 3.51 KiB | 156.00 KiB/s, done.
From https://github.com/chandrashekar0532/git-challenges
* branch      main      -> FETCH_HEAD
  224202a..adbe9f1  main    -> origin/main
Updating 224202a..adbe9f1
Fast-forward

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git pull origin main
From https://github.com/chandrashekar0532/git-challenges
* branch      main      -> FETCH_HEAD
Already up to date.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ |

```

4. Amend a Commit

- Make a commit, then add a missing file to it using `git commit --amend`.

Create file

`echo "Application configuration" > config.txt`

git add config.txt

git commit -m "Add configuration"

Realize another file was forgotten

echo "Application documentation" > docs.txt

git status

Add it to the previous commit

```
MINGW64;f/git-practice
HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ echo "Application configuration" > config.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git commit -m "Add configuration"
[main d413440] Add configuration
 1 file changed, 1 insertion(+)
 create mode 100644 config.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ echo "Application documentation" > docs.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        docs.txt
        git
        git-challenges/
        github
        github-task/

nothing added to commit but untracked files present (use "git add" to track)

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git log --oneline -3
d413440 (HEAD -> main) Add configuration
adb9f1 (origin/main) Merge pull request #4 from chandrashekar0532/developer1
7f01cf9 Merge pull request #3 from chandrashekar0532/main

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ |
```

5. Cherry-pick a Commit

- Take a specific commit from one branch and apply it to another branch.

git switch -c feature-login

echo "Login feature" > login.txt

git add login.txt

git commit -m "Add login feature"

git log --oneline -1

c654f77 Add login feature

Switch master

git switch master

git cherry-pick c654f77

git log --oneline -5

```
HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git switch -c feature-login
Switched to a new branch 'feature-login'

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ echo "Login feature" > login.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ git add login.txt
warning: in the working copy of 'login.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ ls
README.md  a  b  c  config.txt  d  docs.txt  f  g  git  git-challenges/  github  github-task/  h  i  login.txt  readme.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ git commit -m "Add login feature"
[feature-login d3cd25b] Add login feature
1 file changed, 1 insertion(+)
 create mode 100644 login.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ git log --oneline -1
d3cd25b (HEAD -> feature-login) Add login feature

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (feature-login)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git cherry-pick d3cd25b
[main 90ce8c4] Add login feature
 Date: Thu Sep 17 18:20:34 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git log --oneline -5
90ce8c4 (HEAD -> main) Add login feature
d413440 Add configuration
adb9f1 (origin/main) Merge pull request #4 from chandrashekar0532/developer1
7f01cf9 Merge pull request #3 from chandrashekar0532/main
e0e1216 Merge pull request #2 from chandrashekar0532/developer1

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ |
```

6. Interactive Rebase

- Reorder and squash multiple commits into a single clean commit.

git switch -c rebase-demo

1.

echo "Feature code" > feature.txt

git add feature.txt

git commit -m "Add feature"

2.

```
echo "More feature code" >> feature.txt
```

```
git add feature.txt
```

```
git commit -m "fix feature"
```

3.

```
echo "Final feature code" >> feature.txt
```

```
git add feature.txt
```

```
git commit -m "small fix"
```

View history

```
git log --oneline -3
```

Start interactive rebase

```
git rebase -i HEAD~3
```

```
pick 1111111 Add feature
```

```
squash 2222222 fix feature
```

```
squash 3333333 small fix
```

Verify

```
git log --oneline -5
```

```
HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (main)
$ git switch -c rebase-demo
Switched to a new branch 'rebase-demo'

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ echo "Feature code" > feature.txt

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ ls
README.md a b c config.txt d docs.txt f feature.txt g git git-challenges/ github github-task/ h i login.txt readme.txt

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git commit -m "Add feature"
[rebase-demo 131e521] Add feature
1 file changed, 1 insertion(+)
create mode 100644 feature.txt

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ echo "More feature code" >> feature.txt

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git commit -m "fix feature"
[rebase-demo f19639c] fix feature
1 file changed, 1 insertion(+)

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ echo "Final feature code" >> feature.txt

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git commit -m "small fix"
[rebase-demo 0990a6d] small fix
1 file changed, 1 insertion(+)

HP@DESKTOP-IGQ065F MINGW64 /f/git-practice (rebase-demo)
$ git log --oneline -3
0990a6d (HEAD -> rebase-demo) small fix
f19639c fix feature
131e521 Add feature
```

```

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git commit -m "fix feature"
[rebase-demo f19639c] fix feature
1 file changed, 1 insertion(+)

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ echo "Final feature code" >> feature.txt

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git commit -m "small fix"
[rebase-demo 0990a6d] small fix
1 file changed, 1 insertion(+)

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git log --oneline -3
0990a6d (HEAD -> rebase-demo) small fix
f19639c fix feature
131e521 Add feature

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git rebase -i HEAD~3
[detached HEAD a175fa1] Add feature
Date: Thu Sep 17 18:28:18 2026 +0530
1 file changed, 3 insertions(+)
create mode 100644 feature.txt
Successfully rebased and updated refs/heads/rebase-demo.

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git log --oneline -5
a175fa1 (HEAD -> rebase-demo) Add feature
90ce8c4 (main) Add login feature
d413440 Add configuration
adbe9f1 (origin/main) Merge pull request #4 from chandrashekar0532/developer1
7f01cf9 Merge pull request #3 from chandrashekar0532/main

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ |

```

7. Tagging & Release

- Create a version tag (v1.0), push it to GitHub, then delete and restore it.

git switch main

git tag -a v1.0 -m "Release version 1.0"

git tag

Inspect tag

git show v1.0

Push tag

git push origin v1.0

git push origin --tags

Delete local tag

git tag -d v1.0

git tag

Delete remote tag

git push origin --delete v1.0

Restore tag

git tag -a v1.0 -m "Release version 1.0" HEAD

git push origin v1.0

Tagging Flow:

Commit → Tag v1.0 → Push tag → GitHub Release

```
MINGW64:/f/git-practice

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (rebase-demo)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git tag -a v1.0 -m "Release version 1.0"

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git tag
v1.0

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git show v1.0
tag v1.0
Tagger: chandrashekar <psschandrashekar@gmail.com>
Date: Thu Sep 17 18:39:01 2026 +0530

Release version 1.0

commit 90ce8c4a1f64d4593cf423ab96cad2537f24ee4c (HEAD -> main, tag: v1.0)
Author: chandrashekar <psschandrashekar@gmail.com>
Date: Thu Sep 17 18:20:34 2026 +0530

    Add login feature

diff --git a/login.txt b/login.txt
new file mode 100644
index 0000000..f00f155
--- /dev/null
+++ b/login.txt
@@ -0,0 +1 @@
+Login feature

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git push origin v1.0
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 668 bytes | 668.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/chandrashekar0532/git-challenges.git
 * [new tag]          v1.0 -> v1.0

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git push origin --tags
Everything up-to-date

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was d821cd8)

HP@DESKTOP-1GQO65F MINGW64 /f/git-practice (main)
$ git tag
```

```

MINGW64~/f/git-practice
Author: chandrashekar <psschandrashekar@gmail.com>
Date: Thu Sep 17 18:20:34 2026 +0530

    Add login Feature

diff --git a/login.txt b/login.txt
new file mode 100644
index 0000000..f00f155
--- /dev/null
+++ b/login.txt
@@ -0,0 +1 @@
+login feature

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push origin v1.0
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 668 bytes | 668.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/chandrashekar0532/git-challenges.git
 * [new tag]         v1.0 -> v1.0

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push origin --tags
Everything up-to-date

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was d821cd8)

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git tag

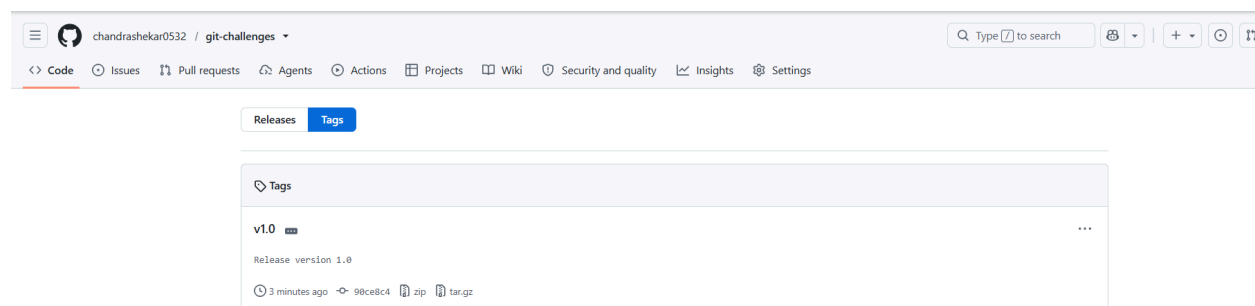
HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push origin --delete v1.0
To https://github.com/chandrashekar0532/git-challenges.git
- [deleted]         v1.0

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git tag -a v1.0 -m "Release version 1.0" HEAD

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git push origin v1.0
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 668 bytes | 668.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/chandrashekar0532/git-challenges.git
 * [new tag]         v1.0 -> v1.0

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ |

```



8. Clone with Sparse Checkout

- Clone only a subdirectory of a repo using sparse checkout.

Clone without checking out files

```
git clone --no-checkout https://github.com/USERNAME/REPOSITORY.git
sparse-lab
```

Enable sparse checkout

git sparse-checkout init --cone

Select directory

git sparse-checkout set backend

Why use sparse checkout?

Useful for:

huge repositories

monorepos

large documentation repositories

working on only one application

```
HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git clone --no-checkout https://github.com/chandrashekar0532/Git-practice.git
Cloning into 'Git-practice'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 18 (delta 6), reused 18 (delta 6), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), done.
Resolving deltas: 100% (6/6), done.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git Git-practice-checkout init --cone
git: 'Git-practice-checkout' is not a git command. See 'git --help'.

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main)
$ git sparse-checkout init --cone

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main|SPARSE)
$ git sparse-checkout set backend

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main|SPARSE)
$ ls
Git-practice/ README.md a b c config.txt d docs.txt f g git git-challenges/ github github-task/ h i login.txt readme.txt

HP@DESKTOP-1GQ065F MINGW64 /f/git-practice (main|SPARSE)
$ |
```

9. Reset vs Revert Challenge

- Demonstrate the difference between `git reset --hard` and `git revert` in a repo.

Part A — git reset --hard

Create:

echo "Version 1" > reset-demo.txt

git add reset-demo.txt

git commit -m "Version 1"

Change it:


```
echo "Version 2" > reset-demo.txt
```

```
git add reset-demo.txt
```

```
git commit -m "Version 2"
```

```
git log --oneline -2
```

```
git reset --hard HEAD~1
```

```
git log --oneline
```

Also: cat reset-demo.txt

Part B — git revert

```
echo "Version 3" > reset-demo.txt
```

```
git add reset-demo.txt
```

```
git commit -m "Version 3"
```

```
git revert HEAD
```

```
git log --oneline -3
```

Command What it does History

git reset --hard Moves branch backward Rewrites history

git revert Creates reverse commit Preserves history

```
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ echo "Version 2" > reset-demo.txt
git commit -m "Version 2"
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git add reset-demo.txt
warning: in the working copy of 'reset-demo.txt', LF will be replaced by CRLF the next time Git touches it

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git commit -m "Version 2"
[main 21ac425] Version 2
1 file changed, 1 insertion(+), 1 deletion(-)
```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git log --oneline -2
21ac425 (HEAD -> main) Version 2
31263eb Version 1

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git reset --hard HEAD~1
HEAD is now at 31263eb Version 1

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git log --oneline
31263eb (HEAD -> main) Version 1
ce895ec (origin/main, origin/HEAD) winter
7ce517c summer
1c9c71a spring collection
ddd626b summer collection updated
6104e67 winter
1a51492 Delete .sass-cache/23fb10bb6531a16595cc9a8cbcaeb8f40f41ec84 directory
bfb08c6 ecommerce

```

```

[main 6cd17cb] Revert "Version 3.3333"
1 file changed, 1 insertion(+), 1 deletion(-)

```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ |

```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git log --oneline -3
6cd17cb (HEAD -> main) Revert "Version 3.3333"
a2a6cdf Version 3
31263eb Version 1

```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ |

```

10. Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it.

git log --oneline

Checkout a specific commit

git checkout 6cd17cb

You are in 'detached HEAD' state

`git status`

What does detached HEAD mean?

Normally:

main → commit C

HEAD → main

Detached:

HEAD → commit B

main → commit C

HEAD is pointing directly to a commit instead of a branch.

GIT CHALLENGE - 3

Create a branch

`git switch -c recovered-feature`

`git branch`

```
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git log --oneline
6cd17cb (HEAD -> main) Revert "Version 3.3333"
a2a6cdf Version 3
31263eb Version 1
ce895ec (origin/main, origin/HEAD) winter
7ce517c summer
1c9c71a spring collection
ddd626b summer collection updated
6104e67 winter
1a51492 Delete .sass-cache/23fb10bb6531a16595cc9a8cbcaeb8f40f41ec84 directory
bfb08c6 ecommerce
```

```
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (main|SPARSE)
$ git checkout 6cd17cb
Note: switching to '6cd17cb'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 6cd17cb Revert "Version 3.3333"

```
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce ((6cd17cb...)|SPARSE)
$ |
```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce ((6cd17cb...)|SPARSE)
$ git status
HEAD detached at 6cd17cb
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce ((6cd17cb...)|SPARSE)
$ git switch -c recovered-feature
Switched to a new branch 'recovered-feature'

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git switch -c recovered-feature
fatal: a branch named 'recovered-feature' already exists

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git branch
  main
* recovered-feature

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ |

```

11. Git Hooks Challenge

- Configure a `pre-commit` hook to reject commits without a message format (e.g., must start with `JIRA-XXX`).

Git Hooks Challenge

Go to Git hooks

`cd .git/hooks`

`ls`

Create pre-commit hook

`nano commit-msg`

`#!/bin/bash`

`commit_message=$(head -n 1 "$1")`

`if [[ ! "$commit_message" =~ ^JIRA-[0-9]+ ]]; then`

`echo "ERROR: Commit message must start with JIRA-XXX"`

`echo "Example: JIRA-123 Fix login issue"`

`exit 1`

Fi

exit 0

chmod +x commit-msg

ls -l commit-msg

Test invalid commit

cd ../../

echo "test" > hook.txt

git add hook.txt

git commit -m "Fix something"

it should reject the commit:

ERROR: Commit message must start with JIRA-XXX

Test valid commit

git commit -m "JIRA-123 Fix something"

```
HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce/.git/hooks (GIT_DIR!|SPARSE)
$ nano commit-msg

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce/.git/hooks (GIT_DIR!|SPARSE)
$ ls -l commit-msg
-rwxr-xr-x 1 HP 197121 223 Aug 31 17:05 commit-msg*

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce/.git/hooks (GIT_DIR!|SPARSE)
$ cd ../../

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ echo "test" > hook.txt

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git add hook.txt
warning: in the working copy of 'hook.txt', LF will be replaced by CRLF the next time Git touches it

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git commit -m "Fix something"
ERROR: Commit message must start with JIRA-XXX
Example: JIRA-123 Fix login issue

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ |
```

```

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce/.git/hooks (GIT_DIR!|SPARSE)
$ cd ../../

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ echo "test" > hook.txt

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git add hook.txt
warning: in the working copy of 'hook.txt', LF will be replaced by CRLF the next time Git touches it

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git commit -m "Fix something"
ERROR: Commit message must start with JIRA-XXX
Example: JIRA-123 Fix login issue

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ git commit -m "JIRA-123 Fix something"
[recovered-feature 348088b] JIRA-123 Fix something
1 file changed, 1 insertion(+)
create mode 100644 hook.txt

HP@Ashish MINGW64 ~/git-advanced-lab/ecommerce (recovered-feature|SPARSE)
$ |

```

12. Squash Merge vs Rebase Merge

- Show the difference between squash merge and rebase merge with Part A — Squash Merge
- git switch main
- git switch -c squash-demo
- Make three commits:
- echo "A" > squash.txt
- git add squash.txt
- git commit -m "Add A"
- echo "B" >> squash.txt
- git add squash.txt
- git commit -m "Add B"
- GIT CHALLENGE - 3
- echo "C" >> squash.txt
- git add squash.txt
- git commit -m "Add C"
- git switch main
- git merge --squash squash-demo
- git commit -m "Add squash demo feature"
- git log --oneline --graph --all
- Part B — Rebase Merge
- Create another branch:
- git switch -c rebase-merge-demo
- echo "X" > rebase.txt
- git add rebase.txt
- git commit -m "Add X"
- echo "Y" >> rebase.txt
- git add rebase.txt
- git commit -m "Add Y"

- git rebase master
- git rebase main
- git merge rebase-merge-demo
- git log --oneline --graph --all
 - evidence.

```
HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ echo "C" >> squash.txt

HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ git add squash.txt
warning: in the working copy of 'squash.txt', LF will be replaced by

HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ git commit -m "Add C"
[squash-demo e46d6c9] Add C
1 file changed, 1 insertion(+)

HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ git switch master
fatal: invalid reference: master

HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ git branch master

HP@Ashish MINGW64 ~/Documents/GIT (squash-demo)
$ git switch master
Switched to branch 'master'

HP@Ashish MINGW64 ~/Documents/GIT (master)
$ git merge --squash squash-demo
Already up to date. (nothing to squash)

HP@Ashish MINGW64 ~/Documents/GIT (master)
$ git commit -m "Add squash demo feature"
On branch master
nothing to commit, working tree clean

HP@Ashish MINGW64 ~/Documents/GIT (master)
$ git log --oneline --graph --all
* e46d6c9 (HEAD -> master, squash-demo) Add C
* d1c2143 Add B
* 02eae9 Add A
```



```

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git commit -m "Add X"
[rebase-merge-demo 06de9da] Add X
1 file changed, 1 insertion(+)
create mode 100644 rebase.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ echo "Y" >> rebase.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time you commit.

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git commit -m "Add Y"
[rebase-merge-demo 92e3c91] Add Y
1 file changed, 1 insertion(+)

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git rebase master
Current branch rebase-merge-demo is up to date.

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git rebase master
Current branch rebase-merge-demo is up to date.

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git merge rebase-merge-demo
Already up to date.

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git merge rebase-merge-demo
Already up to date.

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git log --oneline --graph --all
* 92e3c91 (HEAD -> rebase-merge-demo) Add Y
* 06de9da Add X
* e46d6c9 (squash-demo, master) Add C
* d1c2143 Add B
* 02eaae9 Add A

```

13. Fork & Pull Request Workflow

git clone <https://github.com/chandrashekar0532/ecommm.git>

cd ecomm/

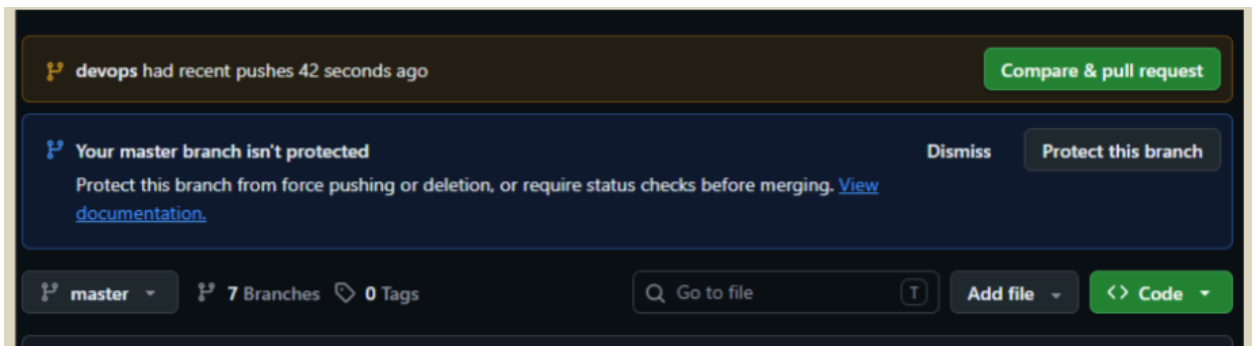
Add original repository as upstream

git remote add upstream <https://github.com/chandu532/ecommm.git>

git remote -v

git switch -c master

```
echo "My contribution" >> README.md
git switch -c master
git add README.md
git commit -m "JIRA-123 Update documentation"
git commit -m "JIRA-123 Update documentation"
```



14.

- Fork a repo, make a change, and submit a pull request to the original repo.

```
echo "Important data" > lost.txt
git add lost.txt
git commit -m "Important commit"
git log --oneline -3
Record the commit
git rev-parse HEAD
Reset hard
git reset --hard HEAD~1
git log --oneline
ls
Use reflog
git reflog
Recover it
```

git switch -c recovered-commit eddd832

Move current branch back

If you know you want to restore the branch:

git reset --hard eddd832

git log --oneline

```
HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ echo "Important data" > lost.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git add lost.txt
warning: in the working copy of 'lost.txt', LF will be replaced by CRLF the

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git commit -m "Important commit"
[rebase-merge-demo e3490cf] Important commit
1 file changed, 1 insertion(+)
create mode 100644 lost.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git log --oneline -3
e3490cf (HEAD -> rebase-merge-demo) Important commit
02e3c91 Add Y
06de9da Add X

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git rev-parse HEAD
e3490cf2decf266368fa8161995e77eab25424dc

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ |
```

```
HP@Ashish MINGW64 ~/Documents/GIT/ecommerce (master)
$ git rev-parse HEAD
eddd832348a322eaf223f5d18048b980a5854d24

HP@Ashish MINGW64 ~/Documents/GIT/ecommerce (master)
$ git reset --hard HEAD~1
HEAD is now at 1ddb518 JIRA-123 Update documentation

HP@Ashish MINGW64 ~/Documents/GIT/ecommerce (master)
$ git log --oneline
1ddb518 (HEAD -> master, origin/master) JIRA-123 Update documentation
f11917e JIRA-123 Update documentation
ce895ec (origin/main, origin/HEAD, main) winter
7ce517c summer
1c9c71a spring collection
ddd626b summer collection updated
6104e67 winter
1a51492 Delete .sass-cache/23fb10bb6531a16595cc9a8cbcaeb8f40f41ec84 directory
bfb08c6 ecommerce
```

```
HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git rev-parse HEAD
e3490cf2decf266368fa8161995e77eab25424dc

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git reset --hard HEAD~1
HEAD is now at 92e3c91 Add Y

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git log neline
fatal: ambiguous argument 'neline': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ ls
ecommerce/ rebase.txt squash.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ |
```

```

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ ls
ecomm/ rebase.txt squash.txt

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git reflog
92e3c91 (HEAD -> rebase-merge-demo) HEAD@{0}: reset: moving to HEAD~1
e3490cf HEAD@{1}: commit: Important commit
92e3c91 (HEAD -> rebase-merge-demo) HEAD@{2}: commit: Add Y
06de9da HEAD@{3}: commit: Add X
e46d6c9 (squash-demo, master) HEAD@{4}: checkout: moving from master to rebase-merge-demo
e46d6c9 (squash-demo, master) HEAD@{5}: checkout: moving from squash-demo to master
e46d6c9 (squash-demo, master) HEAD@{6}: commit: Add C
d1c2143 HEAD@{7}: commit: Add B
02eae9 HEAD@{8}: commit (initial): Add A

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ |

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git reflog
92e3c91 (HEAD -> rebase-merge-demo) HEAD@{0}: reset: moving to HEAD~1
e3490cf HEAD@{1}: commit: Important commit
92e3c91 (HEAD -> rebase-merge-demo) HEAD@{2}: commit: Add Y
06de9da HEAD@{3}: commit: Add X
e46d6c9 (squash-demo, master) HEAD@{4}: checkout: moving from master to rebase-merge-demo
e46d6c9 (squash-demo, master) HEAD@{5}: checkout: moving from squash-demo to master
e46d6c9 (squash-demo, master) HEAD@{6}: commit: Add C
d1c2143 HEAD@{7}: commit: Add B
02eae9 HEAD@{8}: commit (initial): Add A

HP@Ashish MINGW64 ~/Documents/GIT (rebase-merge-demo)
$ git switch -c recovered-commit e3490cf
Switched to a new branch 'recovered-commit'

```

Recover Lost Commit

- Commit something, reset hard, and then recover it using `git reflog`.